

Lab # 02

(Duration: 6 hours)

Lab Reports are to be created using "LibreOffice" and submitted in PDF form at the completion of the lab. You have to create and to submit a single report for this lab.

Submission deadline: By the end of the second lab session.

Answers to the questions/exercises must be illustrated with commands and their outputs. Screenshots must be included in your report. When necessary, you can use existing text files or create your own files.

Objectives of this lab are:

- Continue practicing most used Linux commands.
- Become familiar with keyboard shortcuts and wild cards.
- Learn about redirection and pipes.
- Manipulate text files including searching within their content.
- Compress files.
- Produce the lab report.

1. File/directory management.

The terminal will start in your home directory, `/home/username/`, which is a special directory assigned to your user account. You have already worked with `ls`, `pwd`, `cd`, `mkdir` and `rmdir` commands.

- Use these commands to create a subdirectory `lab2` in which files will be created and processed.
- Create a file `file1.txt` using the command `touch`.
- Write three lines of text in `file1.txt` using `vi` and save the file.
- Create a sub-directory `dir1` and copy `file1.txt` into it.
- Rename the file `file1.txt` of `dir1` to `d1file1.txt`
- Create a second sub-directory `dir2` in the directory `lab2`.
- Create 2 files and a subdirectory in `dir2`.

- Copy the directory `dir2` (including its content) into the directory `dir1`. What is the required option here? Explain.
- Remove `dir2`. What is the required option? Explain.

Command `ls`

- Display the contents of `/usr/local/`
- What is the size of your `.bashrc`?
- What is the `-R` option of command `ls` used for?
- In what order are files and directories arranged (displayed) by default?
- What options can be used to modify this classification criterion?
- Display the contents of your current directory so that most recently modified files appear first.

Directory management

- Is it possible to make a copy of a directory and all of its subdirectories in a single command line?
- What happens if you want to make a copy of a directory `dir1` to a directory `dir2` directory, and the `dir2` directory already exists?
- Is it possible to type a command:

```
cp file1 file2 file3 directory/
```

What is the effect of this line?

- The syntax of `mv` is `mv file1 file2`
⇒ `file1` is renamed to `file2`. But what happens if `file2` already exists?
- Suppose that you have the following directory at home `~/Rapport/docs/` and a file `report.tex` at home. The current directory is `~/Rapport/docs/`

How can you move `report.tex` in `docs`?

How do you then return this file to its original location?

- How do I rename a `dir1` directory to `dir2`? What happens if the `dir2` directory already exists?
- How do I both move a file and change its name? What happens if a file with that name already exists in the destination directory?

2. Wild Cards

Sometimes when we enter a string, we want part of it to be variable, or a wildcard. A common task is to list all files that end with a given extension, such as .txt. The wildcard functionality, through an asterisk, allows to simply say:

```
$ ls *.txt
```

The wildcard can represent a string of any length consisting of any characters - including the empty string.

- What is the other character that can be used to replace any character?
- Illustrate its usage through 2 examples.

3. Useful keyboard shortcuts

Used at the Linux prompt, the keyboard shortcut `Ctrl-P` will roll back to the previous command. If you type `Ctrl-P` twice, you will roll back by two commands. If you type `Ctrl-P` too many times, you can use `Ctrl-N` to move forward. You can also use the arrow keys: up for previous (backward), down for next (forward).

Here are few more useful shortcuts:

- `Ctrl-A` will move you to the beginning of a line.
- `Ctrl-E` will move you to the end of a line.
- `Ctrl-U` will erase everything from where you are in a line back to the beginning.
- `Ctrl-K` will erase everything from where you are to the end of the line.
- `Ctrl-L` will clear the text from current terminal.

Play around with these commands for 5-10 minutes. Being able to scroll back to, edit, and then re-run previously used commands saves time and typing! And like auto-completion, getting in the habit of using keyboard shortcuts will save time.

4. Running Commands Sequentially, Redirection, Piping

It is often convenient to chain together commands that you want to run in sequence. For example, you may want to know in which directory you are and then print the content of directory. This can be done using two commands that are executed in two times.

- How can we execute two or more commands sequentially (in a single command line)?
- Illustrate that through an example in which 3 commands are run sequentially.

Run the following commands in your home directory:

```
$ touch test0.txt
$ touch test1.txt
$ ls > test2.txt
$ cat test2.txt
$ ls -l > test2.txt
$ cat test2.txt
```

- Comment the previous commands.
- Describe the commands and what they do.
- How do we call this operation (>) ?

Here are other commands. Run them.

```
$ ls > test2.txt
$ cat test2.txt
$ ls -l >> test2.txt
$ cat test2.txt
```

- What do you notice. Explain.

What about "<"?

Run the following command:

```
$ wc -c < test2.txt
```

- Explain.

In addition to the ability to direct output to and receive input from files, Linux provides a very powerful capability called *piping*. Piping allows one program to receive as input the output of another program, as follows:

```
$ program1 | program2
```

- Run and explain the following command.

```
$ man -k "directory" | grep "create"
```

5. Basic operations on text files

- Create a text file “numtext.txt”. The file must contain at least 50 lines of text and each line is numbered:

```
1  bla bla bla...
2  bla bla...
3  ...
```

If your solution (to create the file) consists in running several commands, combine and run them *in a single line*. Describe your solution.

- Using the command `less`, print and navigate within the content of the file.
Briefly describe the options of command `less` with your own words.
- Display the 10 first lines of the file.
- Display the 15 last lines of the file.
- Display line number 38.
- Save the 25 first lines of “numtext.txt” to another file (fist25lines.txt).
- Save the lines from 13 to 37 of “numtext.txt” to another file (lines13-37.txt).
- Count and display the number of characters in “lines13-37.txt”.
- Count and display the number of words in “lines13-37.txt”.
- Count and display the number of lines in “lines13-37.txt”.
- Search for a specific string in “numtext.txt”. Display the number of lines where that string appears.

6. Searching files, `grep` command

- Describe the `grep` command and its main options. Use only documentation from command line.
- What are the `grep` options that allow you to obtain context lines (which precede and/or follow the line where the searched word appears)?
- How to display the number of the line where the searched word appears? What happens when we also ask for context lines?
- How to display the number of occurrences of the searched word?
- How to make `grep` ignore character case (difference between upper and lower case) in its search?
- How can I display not the lines where the word appears, but the file names?
- How do I display lines where the search word does not appear?

- How do I display the names of files that do not contain the search word?
- How can I make grep only search lines where the word appears as is, and not its variants? For example: we are looking for the word “work”, but not “worker” or “works”.
- How do I search for several words at once by displaying the line numbers?
- Find all lines starting with “a” or “A”.
- Find all lines ending with “rs”.
- Find all lines containing at least one digit.
- Find all lines starting with a capital letter.
- Search for all lines starting with “B”, “E” or “Q”.
- Find all lines ending with an exclamation point.
- Find all lines that do not end with a punctuation mark (period, comma, semicolon, colon, question mark, exclamation point).
- Search for all words containing an “r” preceded by any upper or lower case letter.
- Find all words with an “r” as the second letter.

7. File compression

All of the following actions must be done in command line.

- Create 3 text files and add text to these files.
- Copy a text file whose size is greater than 10 k bytes from your system
- Download 2 images.
- Store all of these files into one directory named “myDir” under your home.
- Compute the total size of files in this directory.
- Put all the content of the directory in a single uncompressed file (“file1.tar”) and copy it to a sub-directory (“myDir2”).
- Compute the size of the “file1.tar”. What do you notice?
- Compress “file1.tar” and compute its size. What do you notice?
- Decompress the resulting file. Check the produced files and their size. What do you notice.